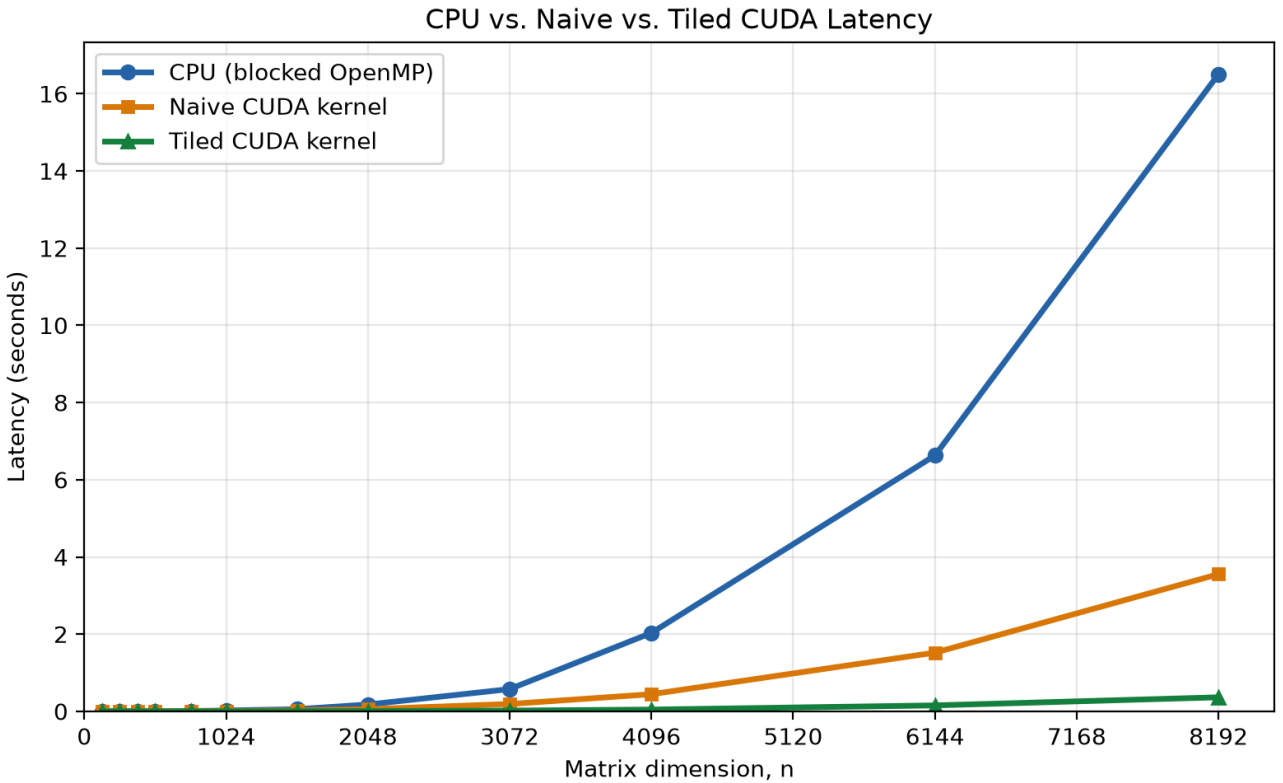
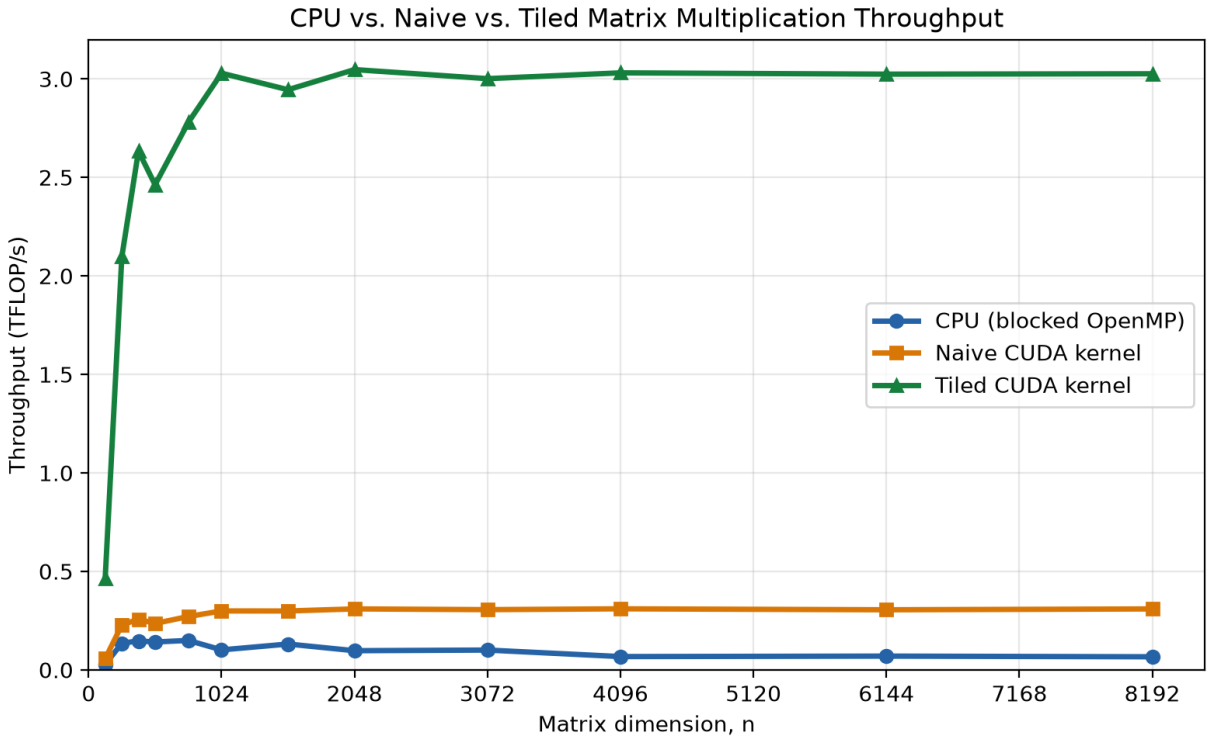
Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.



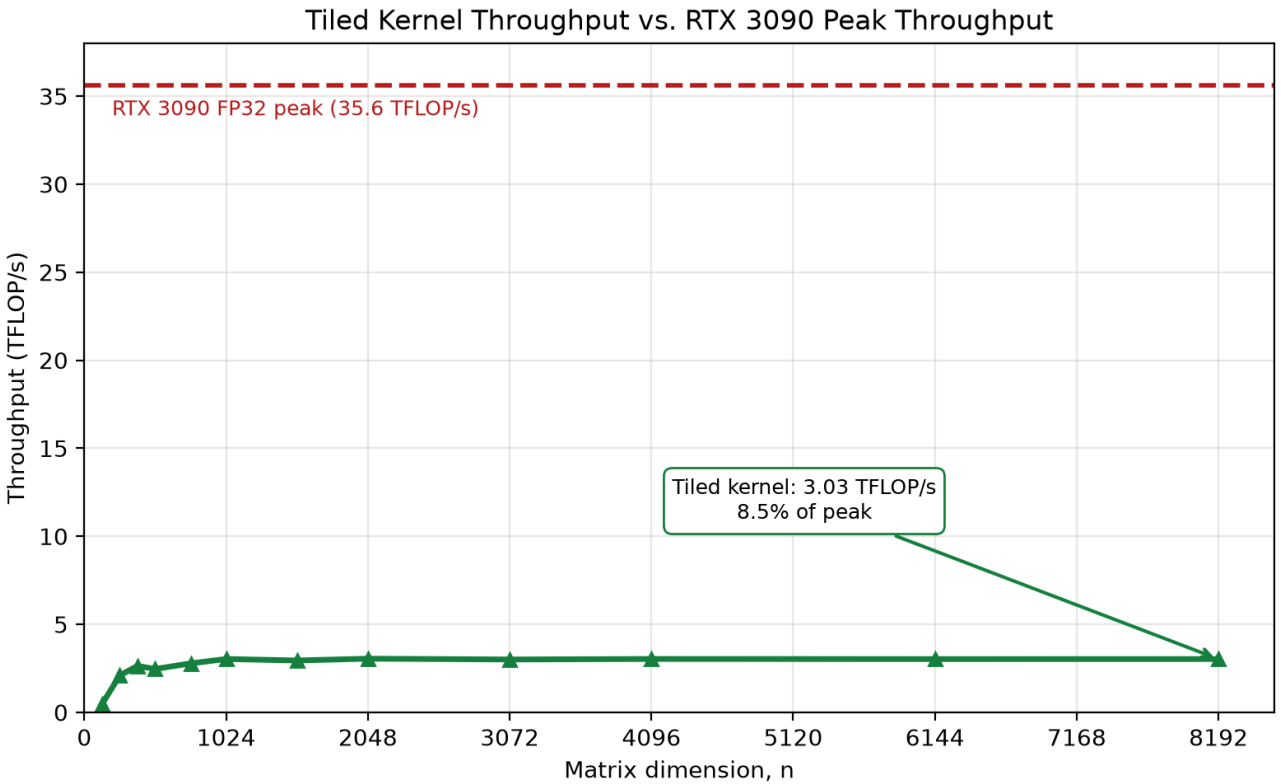
Why latency alone isn't enough

- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.



Why latency alone isn't enough

- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.



Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.
- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.
- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.

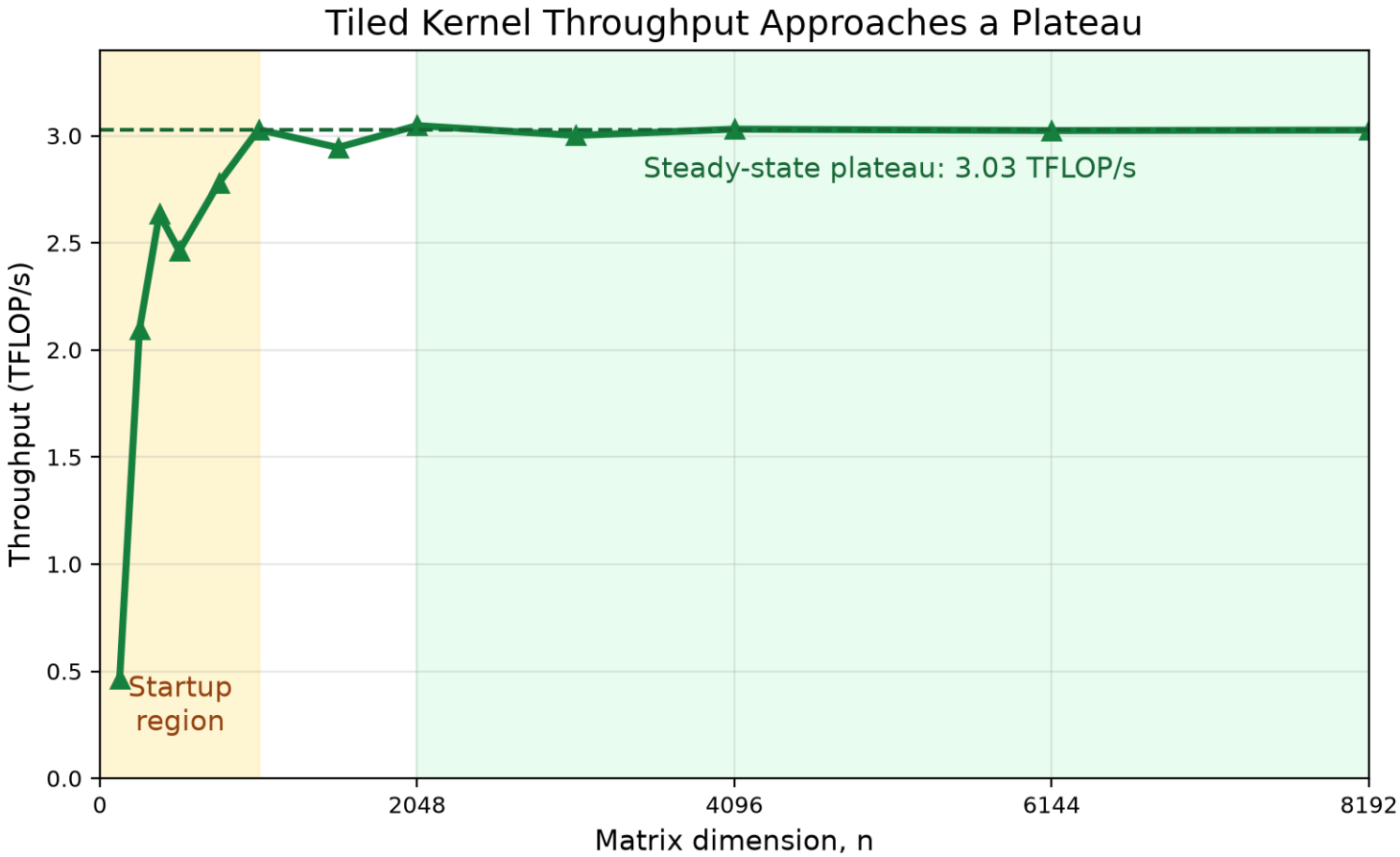
Why throughput

Throughput tells us

1. A general performance metric, no matter the problem size
2. How well we are utilizing hardware

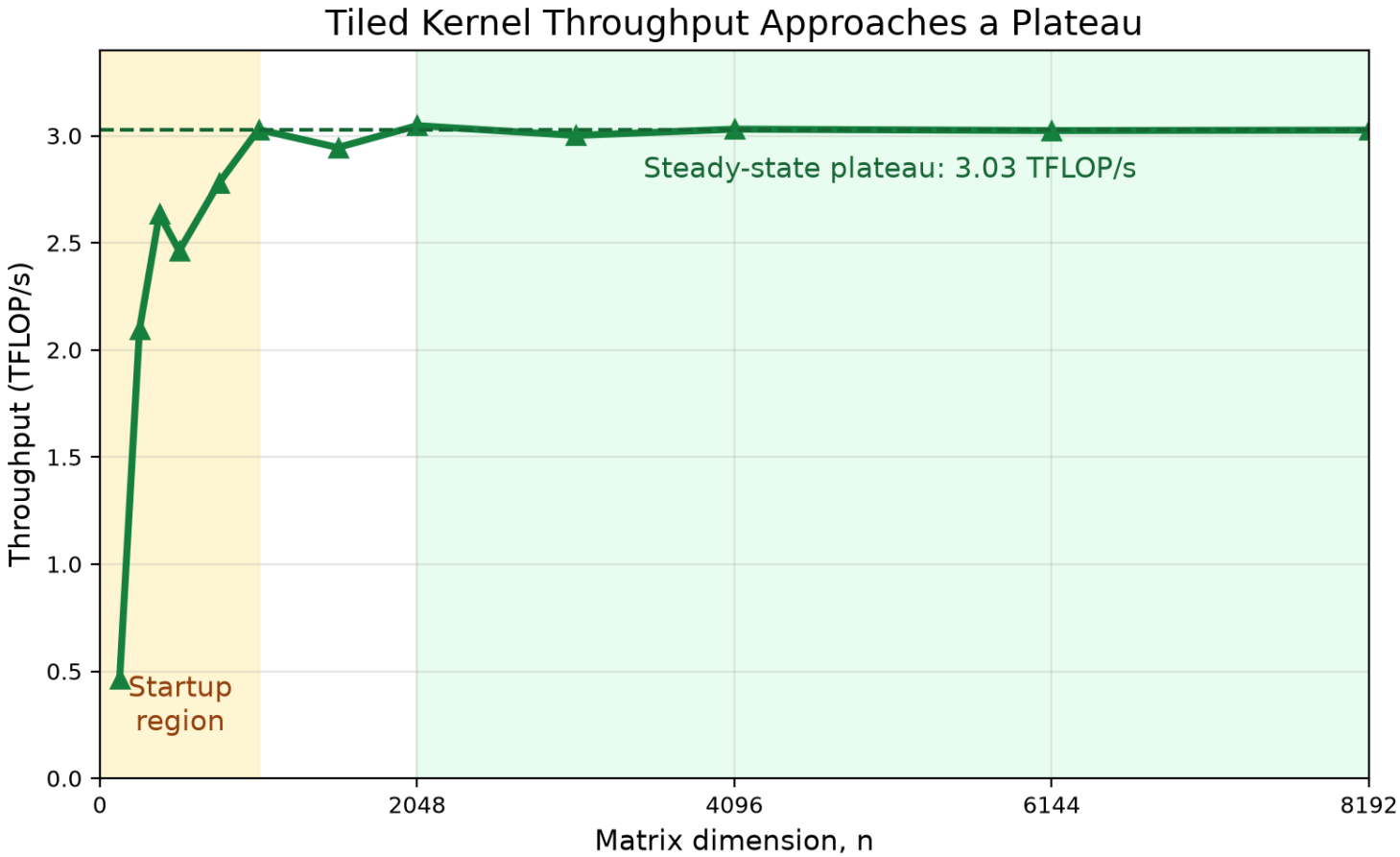
Why throughput still depends on problem size

Normalizing by useful work makes comparisons fairer, not size-independent.



Why throughput still depends on problem size

Normalizing by useful work makes comparisons fairer, not size-independent.



Measurement rule

Measure in the plateau region, and report the problem size with the result.

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example

Peak throughput of our RTX 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs. Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\text{TFLOP/s}.$$

Roughly 8.5% of peak throughput.

Arithmetic Intensity

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example

For $n, k, m = 8192$, our tiled kernel moved a total of 137.7 GB of data from DRAM. With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \times 10^{12} \text{ FLOP}}{137.7 \times 10^{9} \text{ bytes}} \approx \boxed{8.0 \text{ FLOP/byte}}.$$

Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our upper bound throughput is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} = 7.49 \text{ TFLOP/s}.$$

Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our upper bound throughput is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} = 7.49 \text{ TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 TFLOP/s.

Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our upper bound throughput is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} = 7.49 \text{ TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 TFLOP/s. **No matter how we optimize this kernel, without increasing AI, we will never achieve greater than 7.49 TFLOP/s**.

Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth.

Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth. Thus, our ridge point is

$$\text{RP} = \frac{35.6 \times 10^{12}\text{FLOP/s}}{936 \times 10^{9}\text{bytes/s}} \approx 38.0\text{FLOPs/byte}$$

Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth. Thus, our ridge point is

$$\text{RP} = \frac{35.6 \times 10^{12}\text{FLOP/s}}{936 \times 10^{9}\text{bytes/s}} \approx 38.0\text{FLOPs/byte}$$

At AI $= 8.0$ FLOP/byte, we are below the ridge point, and are still memory bound.

Roofline Model of our RTX 3090

